

Universitatea Tehnică „Gheorghe Asachi” din Iași



DroidGuard

Sistem distribuit pentru analiza statică a aplicațiilor Android

Student: Solomon Anastasia

Grupa: 1311B

Cuprins

1	Introducere	3
1.1	Scopul documentului	3
1.2	Scurtă descriere a proiectului	3
2	Descriere generala	4
2.1	Contextul sistemului	4
2.2	Modelarea și persistența datelor	6
2.3	Structura claselor pentru Clientul Mobil	8
2.4	Structura claselor pentru API Gateway	9
2.5	Structura modulelor pentru Analysis Worker	10
2.6	Capabilități generale	12
2.7	Caracteristicile utilizatorului	12
3	Cazuri de utilizare și actori	13
3.1	Actori	13
3.2	Diagrama UML de use-case	13
4	Cerințe funcționale	14
5	Cerințe non-funcționale	16
5.1	Capacitate	16
5.2	Performanță	16
6	Interfața grafică	17
6.1	Dashboard-ul principal și selecția aplicațiilor	17
6.2	Fluxul de inițiere, monitorizare și anularea scanării	18
6.3	Gestionarea erorilor și reîncercarea scanării	20
6.4	Gestionarea istoricului și rapoartele de securitate	21
6.5	Prezentarea rezultatului	22
6.6	Suport multilingv	23
7	Testare	24
7.1	Testarea pe aplicații benigne	24
7.2	Testarea pe aplicații malițioase construite manual	24
7.3	Sinteza rezultatelor	26
7.4	Limitări în analiza aplicațiilor malițioase reale	26
8	Concluzii	27
	Bibliografie	28

Versiuni

Data	Versiune	Modificări
23.06.2026	0.1	Structura inițială a documentului.
24.06.2026	0.2	Adăugarea cerințelor specifice.
25.06.2026	0.3	Adăugarea descrierii generale, integrarea diagramei de arhitectură, definirea capabilităților, actorilor și cerințelor non-funcționale.
26.06.2026	0.4	Redactarea capabilităților, crearea diagramei UML de use-case și de secvență, detalierea cerințelor specifice.
02.07.2026	0.5	Rescrierea secțiunii contextul sistemului. Adăugarea secțiunii de modelare a datelor și a diagramei Entitate-Relație.
03.07.2026	0.6	Crearea diagramelor UML de clase. Adăugarea detaliilor de implementare și a structurii arhitecturale pentru Clientul Mobil și API Gateway.
09.07.2026	0.7	Integrarea bazei de date locale și crearea diagramei de secvențe. Finalizarea structurii arhitecturale pentru Analysis Worker și adăugarea diagramei de clase aferente.
11.07.2026	0.8	Adăugarea secțiunii de design al interfeței grafice, integrarea capturilor de ecran și documentarea fluxurilor de utilizare.
17.07.2026	0.9	Actualizarea secțiunii de design al interfeței grafice cu funcționalității de căutare în ecranul de selecție a aplicațiilor. Documentarea suportului multilingv: română și engleză.
18.07.2026	1.0	Extinderea secțiunii de testare cu evaluarea pe aplicații malițioase construite manual, în trei versiuni cu severitate progresivă.
25.07.2026	1.1	Integrarea logo-ului pe pagina de titlu. Documentarea mecanismelor de gestionare a erorilor și menționarea barei de căutare în istoricul scanărilor.
26.07.2026	1.2	Detalierea aprofundată a mecanismelor tehnice pentru Analysis Worker. Adăugarea limitărilor analizei statice pe malware real și a perspectivelor de utilizare a tehnicilor de Machine Learning.
29.07.2026	1.3	Detalierea mecanismului de pornire a timer-ului și a rulării aplicației în fundal. Documentarea sistemului de notificări și a filtrării dinamice a rapoartelor. Actualizarea imaginilor aferente interfeței.

1 Introducere

1.1 Scopul documentului

Scopul acestui document este de a oferi o descriere a cerințelor funcționale și non-funcționale pentru proiectul DroidGuard, formulate clar din perspectiva utilizatorului și a modului în care acesta interacționează cu sistemul.

1.2 Scurtă descriere a proiectului

Proiectul DroidGuard este un sistem software distribuit, dedicat analizei statice de securitate a aplicațiilor Android.

Scopul principal al platformei este de a oferi utilizatorilor o soluție pentru detectarea posibilelor amenințări.

2 Descriere generala

2.1 Contextul sistemului

Arhitectura sistemului DroidGuard este construită pe un model pe trei niveluri, conceput pentru a separa interfața utilizatorului de procesarea asincrona a fișierelor complexe. Figura 1 ilustrează fluxul de date și componentele principale ale sistemului.

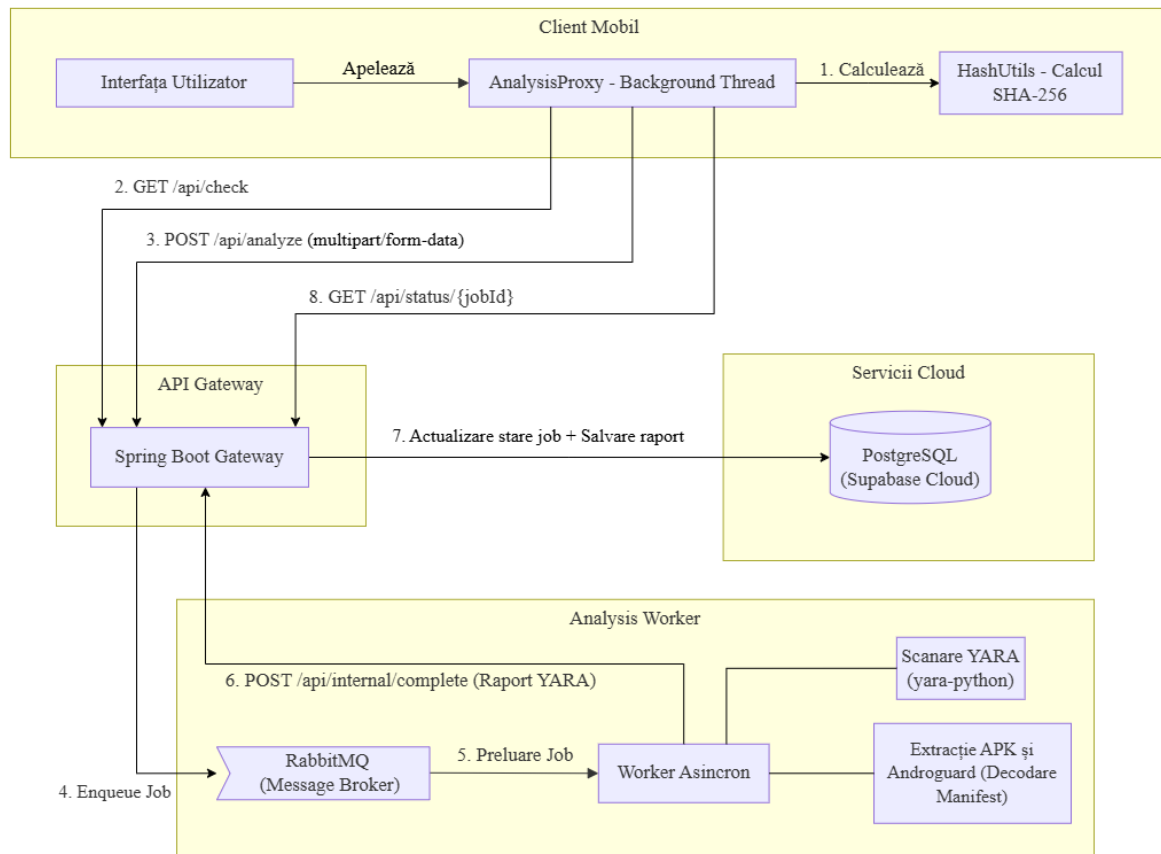


Figura 1: Arhitectura sistemului DroidGuard și fluxul de procesare client-server

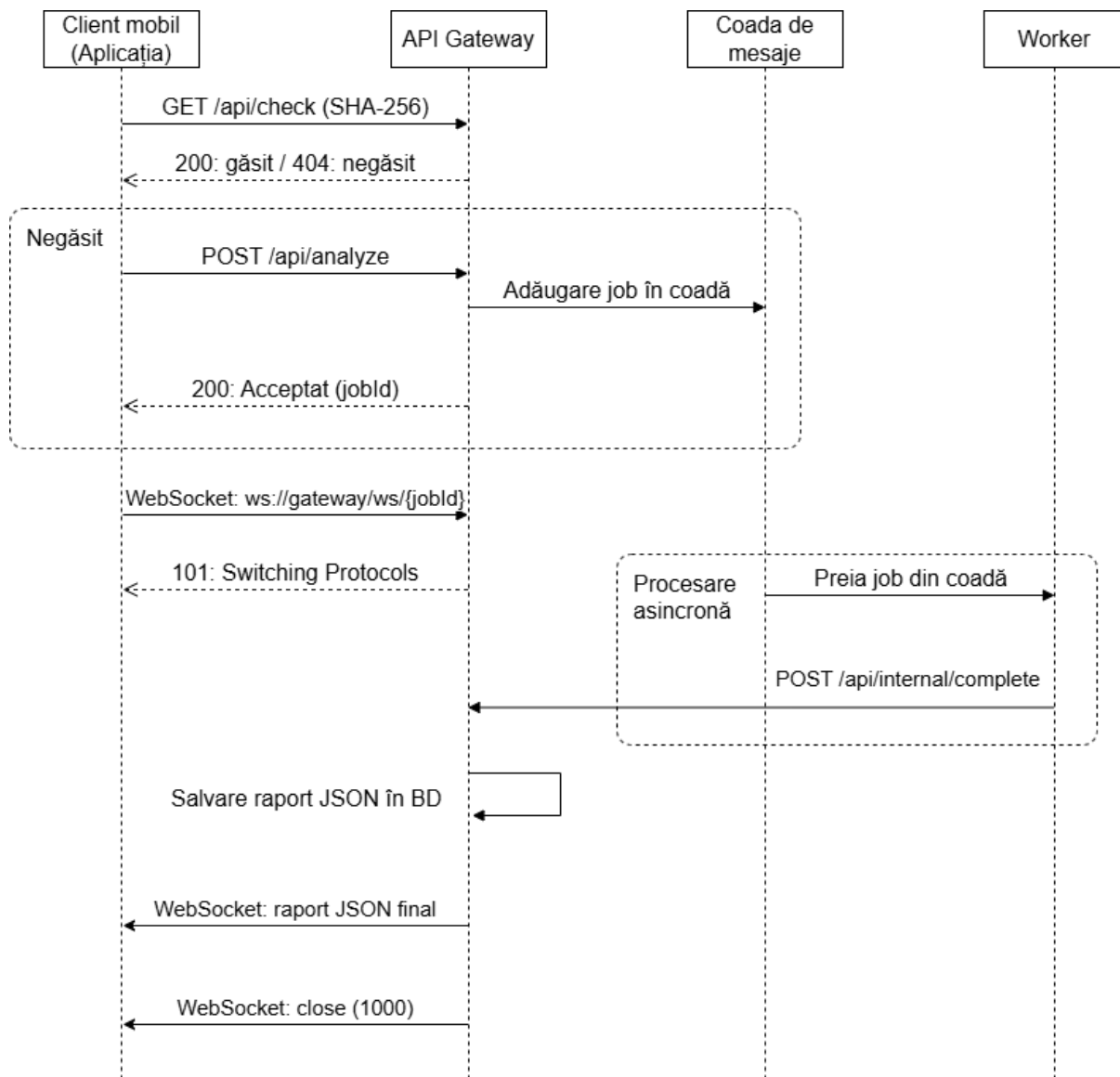


Figura 2: Diagrama de secvență a sistemului DroidGuard

Sistemul DroidGuard are cele trei niveluri funcționale și integrate, după cum urmează:

- **Clientul Mobil (Android / Java):** Gestionează interfața grafică și comunicarea cu serverul. Pentru a menține fluiditatea interfeței și a preveni blocajele de tip ANR, calculul local al amprentei criptografice SHA-256 este executat asincron pe fire de execuție secundare folosind `ExecutorService`. O optimizare critică la nivel de memorie este utilizarea `HttpURLConnection` configurat cu `setChunkedStreamingMode` pentru încărcarea aplicațiilor (prin `multipart/form-data`). Această abordare asigură transferul pachetelor APK de dimensiuni mari fără a epuiza memoria RAM a dispozitivului.
- **API Gateway (Spring Boot / Java):** Acționează ca un orchestrator central pentru rutarea sarcinilor și gestionarea stărilor. Acesta expune endpoint-uri REST pentru verificări, interogări de status și utilizează **RabbitMQ** (model *fire-and-forget*) pentru a publica asincron sarcinile către componenta de execuție. Persistența datelor este izolată în cloud prin platforma Supabase (PostgreSQL).

- **Analysis Worker (Python):** Este componenta autonomă care execută munca intensivă. Aceasta consumă mesajele din RabbitMQ, extrage fișierele APK utilizând biblioteca `zipfile` și folosește **Androguard** pentru a decoda binarul **AndroidManifest** (fișier xml) direct în text lizibil, pregătindu-l pentru detecția de permisiuni malițioase. Scanarea propriu-zisă folosește motorul **yara-python**. Performanța worker-ului este optimizată prin:
 - **Pre-compilarea în memorie:** Regulile YARA sunt compilate global, o singură dată la pornirea serviciului, eliminând latența per-cerere.
 - **Filtrare I/O:** S-a implementat o listă de extensii (`.dex`, `.so`, `.xml` etc.) pentru a exclude de la citire resurse media irelevante incluse în aplicații.
 - **Procesare concurentă:** Prin scanarea pe mai multe fire.

Comunicarea în timp real dintre clientul mobil și serverul Gateway a fost optimizată prin înlocuirea mecanismului inițial de *HTTP polling* cu o conexiune persistentă bazată pe protocolul WebSocket. În abordarea anterioară, clientul trimitea cereri repetate către endpoint-ul `GET /api/status/{jobId}` la intervale regulate pentru a verifica starea analizei. Prin implementarea WebSocket-ului, serverul Gateway notifică clientul în momentul în care raportul final este disponibil, eliminând interogările inutile.

2.2 Modelarea și persistența datelor

Având în vedere arhitectura distribuită a sistemului DroidGuard, persistența datelor este gestionată pe două niveluri interconectate: o bază de date centralizată (pentru componenta Gateway) și o memorie cache locală (pentru aplicația Android client).

Pentru a stoca la nivel global istoricul fișierelor procesate și rezultatele analizelor statice, serverul Gateway utilizează o bază de date relațională (PostgreSQL) aflată în cloud prin platforma Supabase. Figura 3 ilustrează diagrama Entitate-Relație a sistemului DroidGuard în etapa curentă.

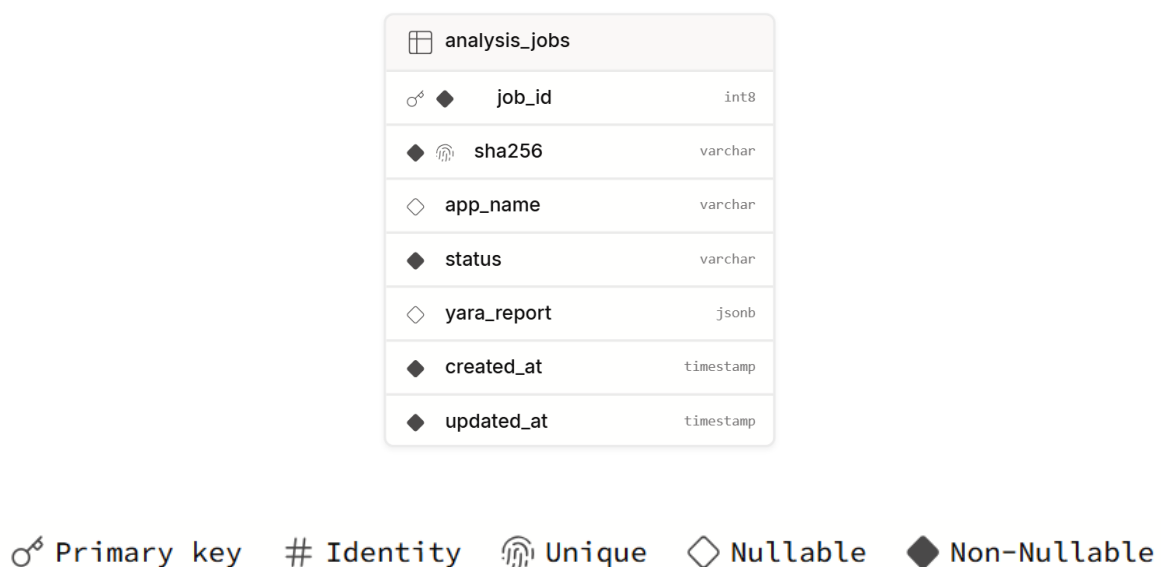


Figura 3: Diagrama E-R a bazei de date cloud și legenda simbolurilor utilizate

În loc să avem o divizare excesivă a tabelor relaționale pentru fiecare tip de vulnerabilitate detectată, rezultatul final este salvat într-o coloană de tip JSONB. Această abordare oferă flexibilitate pentru structura dinamică a rapoartelor generate de YARA și Androguard.

De asemenea, această structură reprezintă o fundație solidă pentru dezvoltarea ulterioară, cum ar fi integrarea unui sistem de gestionare a utilizatorilor. Astfel, un raport de securitate generat în urma unei scanări unice va putea fi asociat ulterior istoricului mai multor utilizatori.

Pe lângă infrastructura din cloud gestionată de serverul Spring Boot, aplicația Android utilizează o bază de date locală (SQLite, gestionată prin biblioteca Room). Figura 4 prezintă schema acestei componente.

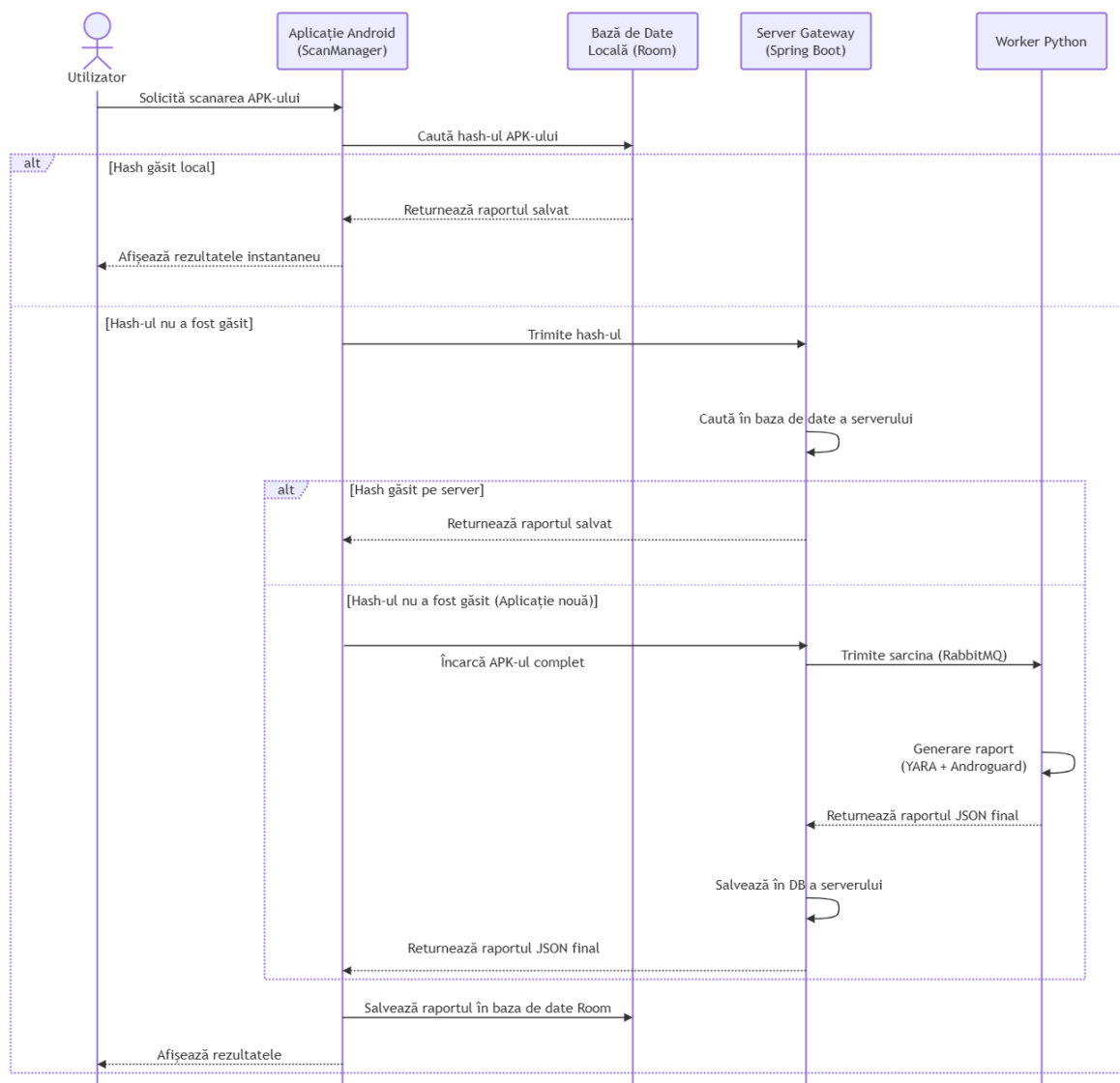


Figura 4: Diagrama de secvență a bazei de date locale utilizată de aplicația Android

Baza de date locală acționează ca un prim nivel de stocare, salvând un istoric al scanărilor efectuate direct pe telefonul utilizatorului. Aceasta permite încărcarea instantanee

a rapoartelor anterioare și elimină necesitatea unor interogări de rețea redundante către server.

2.3 Structura claselor pentru Clientul Mobil

Pentru aplicația Android, arhitectura claselor a fost proiectată cu scopul principal de a separa interfața grafică de operațiunile costisitoare din punct de vedere al prelucrării. Figura 5 ilustrează componentele principale ale clientului mobil. În cadrul diagramei au fost omise clasele, metodele și câmpurile irelevante descrierii generale.

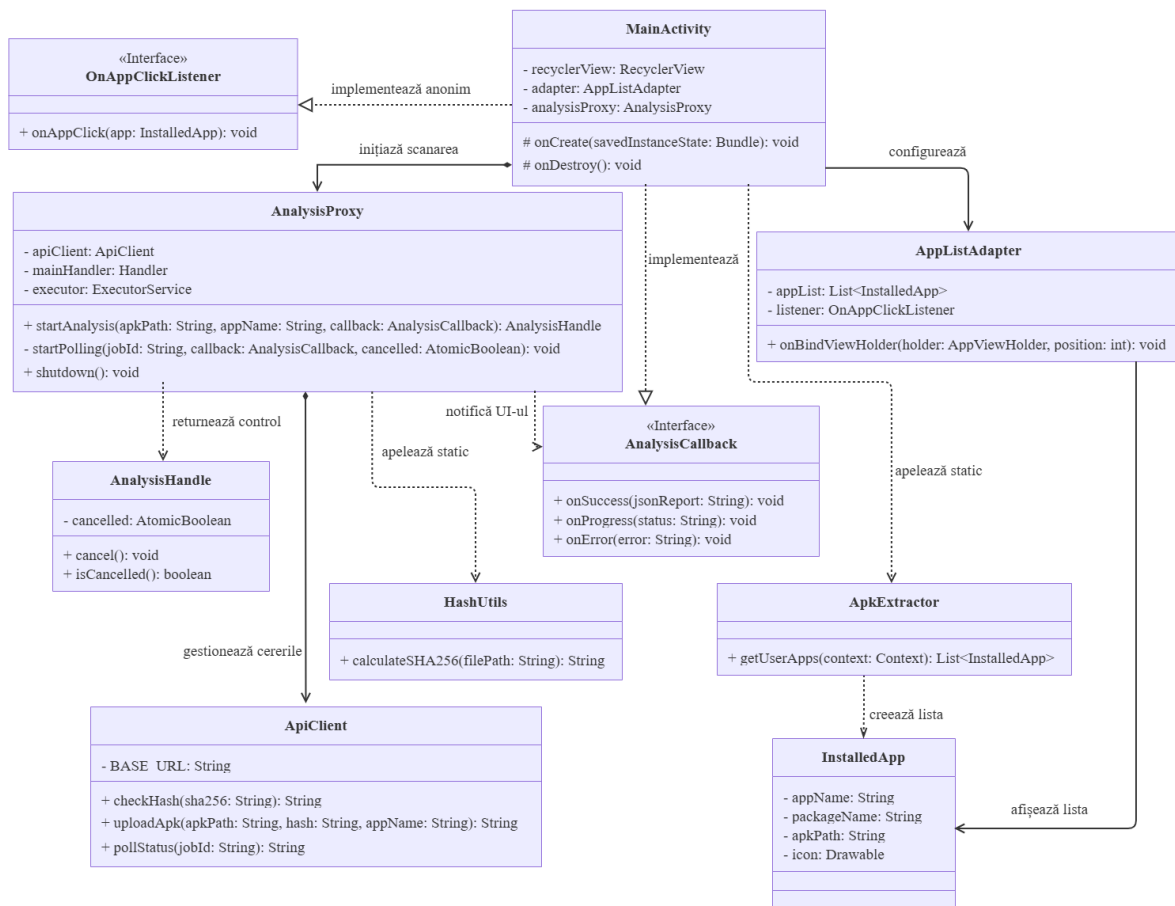


Figura 5: Diagrama principală de clase pentru Clientul Mobil

Clientul este divizat în următoarele componente logice:

- **Nivelul de interfață:** Clasa **MainActivity** este responsabilă pentru interacțiunea cu utilizatorul. Aceasta utilizează clasa utilitară **ApkExtractor** (care interoghează **PackageManager**) pentru a popula lista de obiecte de tip **InstalledApp**. Pentru afișarea dinamică, folosește **AppListAdapter** și implementează interfața **AnalysisCallback** pentru a actualiza ecranul asincron pe baza stării procesării.
- **Nivelul de coordonare asincronă:** Clasa **AnalysisProxy** acționează ca un intermediar. Pentru a preveni erorile de tip ANR, aceasta utilizează un **ExecutorService** pentru a muta calculele și cererile HTTP pe fire de execuție secundare. Aceasta

returnează un obiect **AnalysisHandle**, permițând interfeței să anuleze operațiunea la nevoie.

- **Utilitare și rețea:**

- Clasa **HashUtils** execută citirea secvențială a fișierului APK, utilizând un buffer, pentru generarea amprente SHA-256 locale.
- Clasa **ApiClient** încapsulează logica de comunicare cu serverul. Utilizarea **HttpURLConnection** configurat cu **setChunkedStreamingMode** în metoda **uploadApk** permite transmiterea fișierelor de dimensiuni mari prin **multipart/formdata** fără a le încărca integral în memoria RAM a telefonului.

2.4 Structura claselor pentru API Gateway

Componenta API Gateway a fost implementată respectând modelul specific framework-ului Spring Boot. Figura 6 ilustrează interacțiunile dintre clasele principale ale sistemului, excluzând obiectele de configurare și metodele de acces pentru a evidenția fluxul logic de date.

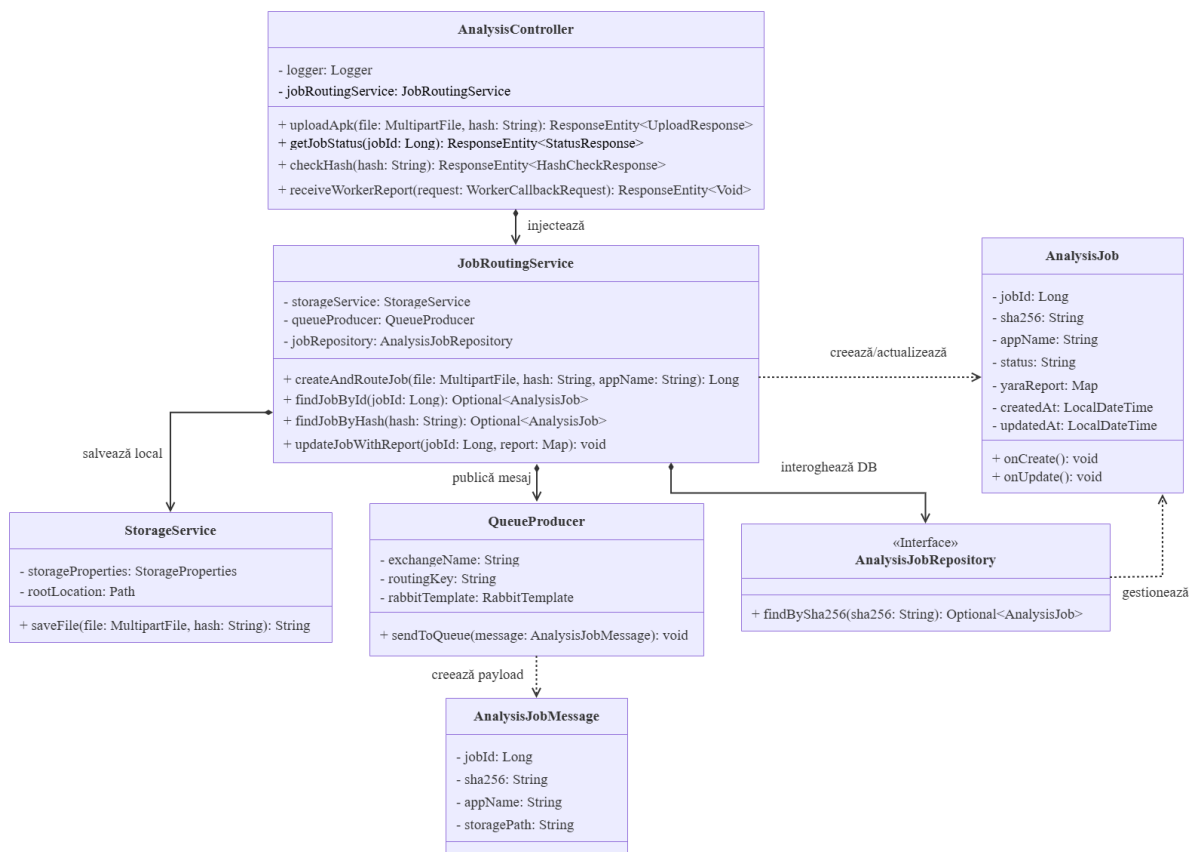


Figura 6: Diagrama principală de clase pentru API Gateway

Sistemul este structurat pe următoarele niveluri de responsabilitate:

- **Nivelul de prezentare:** Clasa **AnalysisController** gestionează exclusiv cererile HTTP externe primite de la clientul mobil și apelurile interne primite de la **Analysis**

Worker. Aceasta validează datele de intrare și delegă execuția către serviciile de business.

- **Nivelul de business:** Clasa `JobRoutingService` reprezintă nucleul logic al aplicației. Aceasta orchestrează întregul proces: interoghează baza de date, apelează `StorageService` pentru persistența locală a fișierelor APK și utilizează `QueueProducer` pentru a crea și publica obiectul `AnalysisJobMessage` în RabbitMQ.
- **Nivelul de persistență:** Interfața `AnalysisJobRepository` extinde funcționalitățile JPA pentru interogarea bazei de date (ex. `findBySha256`), gestionând entitatea `AnalysisJob` care mapează direct structura tabelului din Supabase.

2.5 Structura modulelor pentru Analysis Worker

Componenta Analysis Worker este structurată modular pentru a separa gestionarea sarcinilor de analiză propriu-zisă a fișierelor APK. Figura 7 ilustrează fluxul logic esențial al acestei componente.

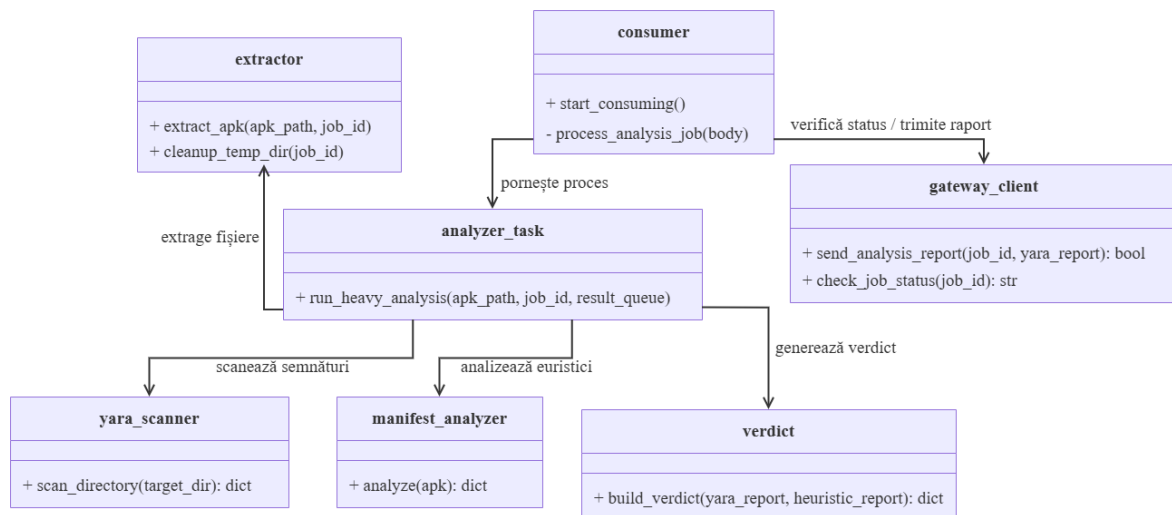


Figura 7: Diagrama principală de clase pentru Analysis Worker

Sistemul este compus din următoarele entități și mecanisme de analiză:

- **Consumer:** Gestionează comunicarea cu broker-ul RabbitMQ, preluând sarcinile de scanare. Acesta verifică statusul curent al sarcinii prin intermediul `GatewayClient` pentru a se asigura că nu a fost anulată de utilizator. Pentru a asigura revenirea sistemului și izolarea defectelor, analiza nu rulează în firul principal, ci este un proces separat. Acest lucru garantează că o eroare critică sau o buclă infinită generată de un malware nu va bloca serviciul de consumare a mesajelor.
- **AnalyzerTask:** Acționează ca orchestrator central apelând secvențial modulele de prelucrare: extracția, scanarea YARA, analiza Androguard și calculul verdictului.
- **Extractor:** Modulul care pregătește mediul de lucru. Utilizează tehnici de adaptare a codului peste biblioteca standard `zipfile` pentru a ignora intenționat erorile de sumă de control și pentru a curăța anteturile de compresie falsificate, eli-

minând tehnicile anti-analiză folosite frecvent de malware pentru a bloca dezasamblarea. De asemenea, utilizează **Androguard** și **lxml** pentru a converti fișierul binar **AndroidManifest.xml** într-un format text lizibil.

- **YaraScanner:** Motorul pentru analiza semnăturilor statice. Pentru a elimina latența la fiecare cerere, regulile YARA sunt validate și pre-compilate global în memorie la inițializarea scanării. Scanarea directorului extras este executată concurrent, bazându-se pe un **ThreadPoolExecutor** cu 10 fire de execuție. Pentru a optimiza consumul de resurse, sunt scanate doar fișierele cu extensii relevante analizei (precum **.dex**, **.so**, **.xml**, **.js**). Motorul extrage și aplică niveluri de încredere pentru fiecare regulă, bazate pe configurația din fișierul **rule_confidence.json**.
- **ManifestAnalyzer:** Modulul euristic central care construiește o analiză peste bytecode-ul Dalvik folosind **Androguard**. Extrage și evaluează o serie de indicatori de compromitere:
 - Identifică combinațiile de permisiuni care facilitează atacuri specifice, precum interceptarea SMS cu persistență la boot care este specific troienilor bancari.
 - Detectează componentele Android exportate (aflate în activity, service, receiver) care expun acțiuni sensibile fără a solicita o permisiune de protecție.
 - Calculează entropia Shannon pentru fiecare fișier **.dex**. Un scor mai mare sau egal cu 7.3 biți/octet semnalează utilizarea tehnicilor de împachetare sau criptare a codului.
 - Generează referințe XREF în bytecode pentru a depista apelurile API invocate în mod malițios, identificând comportamente precum execuția de comenzi la runtime, încărcarea dinamică a claselor, trimiterea programmatică de SMS-uri și accesarea identificatorilor hardware.
 - Identifică tehnicile specifice de interceptare SMS, prin căutarea apelurilor către **abortBroadcast()** sau a receiverelor cu prioritate ridicată artificial.
- **Verdict:** Colectează rezultatele intermediare și calculează scorul final de risc bazându-se pe un model de fuziune prin coroborare.
 - Sistemul organizează descoperirile pe canale independente (semnături, apeluri API, structură, permisiuni, certificate).
 - Un comportament sever necesită confirmare din mai multe canale (ex: permisiune prezentă + apel API efectuat) pentru a acumula punctajul maxim (peste 0.75 pentru *Malicious*). Dacă două sau trei canale confirmă același comportament, se aplică multiplicatori de coroborare progresivi.
 - Tehnicile de evaziune descoperite (cum ar fi ofuscarea sau entropia mare) nu adaugă scor direct, ci acționează ca un amplificator matematic peste amenințările confirmate.
 - În situația în care **YaraScanner** raportează o potrivire pe o regulă cu grad ridicat de încredere, motorul scurtcircuitează logica de calcul și atribuie direct verdictul *Malicious* cu scorul 1.0, raportul explicând explicit motivul deciziei.

2.6 Capabilități generale

Sistemul este conceput pentru a îndeplini următoarele funcții:

- **Extragerea aplicațiilor locale:** Identificarea aplicațiilor instalate pe dispozitivul mobil (filtrând aplicațiile de sistem) și preluarea fișierelor APK fizice corespunzătoare.
- **Eliminarea dublurilor:** Calcularea locală a amprentei SHA-256 a fișierului și interogarea bazei de date de pe server pentru a preveni încărcarea repetată a aplicațiilor deja analizate.
- **Decompilare și traducere:** Transformarea codului binar compilat (Dalvik byte-code / *.dex*) în format text (*.smali*) pe serverul de analiză, oferind astfel structura necesară pentru o scanare eficientă.
- **Scanare deterministă bazată pe reguli (YARA):** Analizarea codului sursă extras pentru a detecta semnături de cod malițios și amenințări cunoscute.
- **Raport asincron:** Generarea unor rapoarte de securitate detaliate în format JSON și livrarea acestora către clientul mobil.

2.7 Caracteristicile utilizatorului

Utilizatorul țintă al platformei DroidGuard este o persoană care deține un dispozitiv mobil cu sistem de operare Android și este interesată de verificarea de securitate a aplicațiilor instalate local. Acesta trebuie să posede doar un nivel minim de cunoștințe specifice utilizării aplicațiilor mobil standard. Sistemul este proiectat astfel încât să ascundă complet complexitatea tehnică în spatele unei interfețe grafice simple.

Cu toate acestea, pentru a beneficia la maximum de funcționalitățile sistemului, utilizatorul va trebui să aibă o înțelegere de bază a conceptelor de securitate cibernetică la citirea rapoartelor finale (de exemplu, să înțeleagă diferența dintre o aplicație sigură și una nesigură).

3 Cazuri de utilizare și actori

3.1 Actori

În cadrul aplicației, au fost identificați următorii actori:

- **Utilizator Android:** Reprezintă persoana fizică ce utilizează aplicația mobilă. Acest actor inițiază procesul de verificare, aprobă încărcarea fișierelor și vizualizează rapoartele de securitate. Interacțiunea sa are loc prin intermediul interfeței grafice.
- **API Gateway:** Reprezintă componenta centrală care primește cererile de la clientul mobil. Acționează ca un intermediar care gestionează interogările bazei de date, acceptă fișierele și delegă sarcinile mai departe.
- **Analysis Worker:** Reprezintă componenta asincronă care execută analiza intensivă. Acționează autonom odată ce preia un mesaj din coada de mesaje, coordonând decompilarea fișierelor și rularea motorului YARA.

3.2 Diagrama UML de use-case

Diagrama de mai jos ilustrează interacțiunile dintre actorii identificați și cazurile de utilizare specifice sistemului DroidGuard.

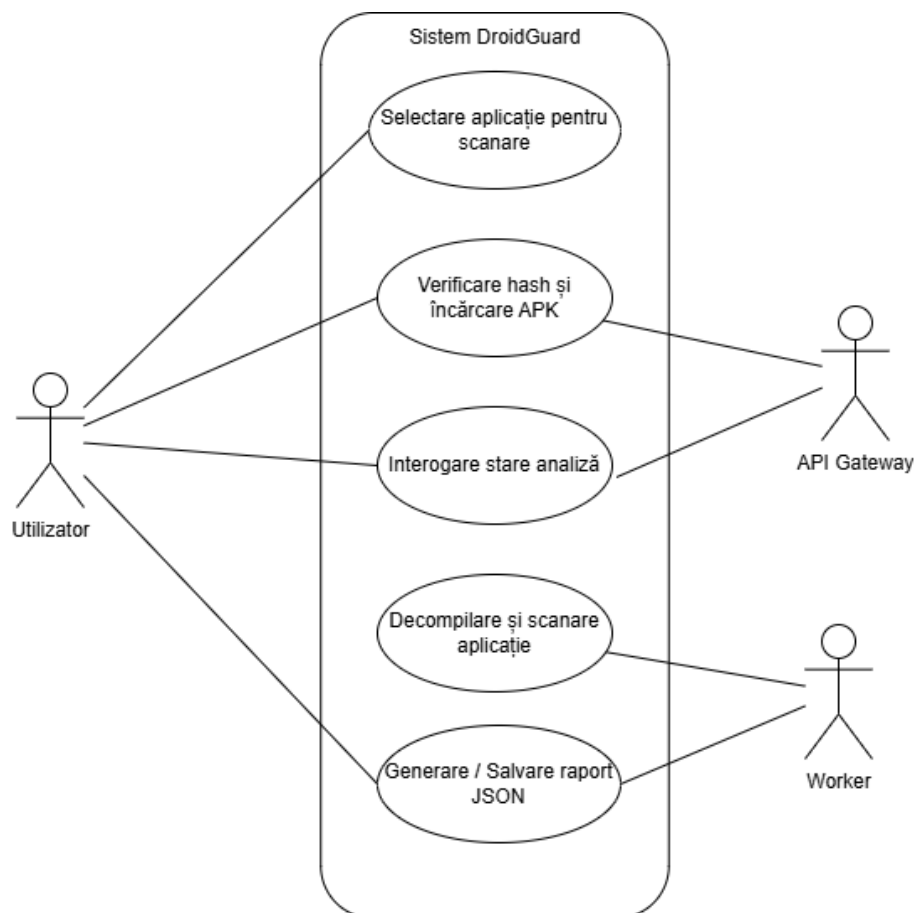


Figura 8: Diagrama de cazuri de utilizare a sistemului DroidGuard

4 Cerințe funcționale

Nr.	Rol	Descrierea cerinței	Motivație	Prioritate
1	Utilizator	Sistemul va afișa o listă a aplicațiilor instalate de utilizator, excluzând pachetele de sistem.	Facilitarea selecției rapide a aplicației țintă.	Maximă
2	Sistem	Calculare și interogare automată a amprentei criptografice SHA-256 a aplicației pe server, anterior oricărui transfer de date.	Optimizarea lățimii de bandă și furnizarea unui răspuns instantaneu în cazul fișierelor existente.	Maximă
3	Utilizator	Aplicația client va permite transferul securizat al fișierului APK fizic către infrastructura Droid-Guard.	Externalizarea procesului de analiză statică.	Maximă
4	Utilizator	Utilizatorul va avea posibilitatea de a monitoriza stadiul de procesare al fișierului transmis.	Asigurarea transparenței privind progresul sarcinii asincrone de decompilare și scanare executate pe server.	Maximă
5	Utilizator	Interfața grafică va prezenta utilizatorului raportul final de securitate într-un format structurat și lizibil.	Furnizarea datelor necesare pentru identificarea amenințărilor detectate, permițând o evaluare informată a riscurilor.	Maximă
6	Sistem	Sistemul va asigura execuția procesului de monitorizare în fundal și va afișa notificări persistente privind starea scanărilor active.	Garantarea continuității analizei asincrone și a conexiunii WebSocket chiar dacă aplicația este minimizată.	Ridică
7	Utilizator	Sistemul va notifica utilizatorul la finalizarea procesului de analiză, oferind acces direct la raportul de securitate.	Îmbunătățirea experienței utilizatorului prin eliminarea necesității de a verifica manual starea aplicației.	Ridică

Nr.	Rol	Descrierea cerinței	Motivație	Prioritate
8	Utilizator	Sistemul va permite filtra- rea și căutarea dinamică a aplicațiilor în istoricul de scanări salvat local.	Facilitarea navigării ra- pide printr-un volum mare de rapoarte și identificarea rapidă a amenințărilor.	Medie

5 Cerințe non-funcționale

Definirea constrângerilor și atributelor de calitate ale sistemului sunt esențiale pentru asigurarea unei experiențe optime pentru utilizator și a unei operări sigure pe server.

5.1 Capacitate

- **Gestionarea concurenței:** Datorită arhitecturii decuplate bazate pe cozi de mesaje, sistemul trebuie să poată prelua și stoca în așteptare un număr teoretic nelimitat de cereri simultane de analiză, evitând pierderea datelor sau suprasolicitarea Gateway-ului în perioadele de vârf.
- **Dimensiunea maximă a fișierelor:** API Gateway-ul trebuie să permită transferul unor fișiere APK cu o dimensiune de până la 100 MB, acoperind astfel majoritatea aplicațiilor standard de Android.
- **Scalabilitate orizontală:** Arhitectura componentei de backend trebuie să permită adăugarea facilă de noi instanțe pentru a mări capacitatea globală de analiză a sistemului, fără a necesita modificări la nivelul codului sursă.

5.2 Performanță

- **Timp de răspuns sincron:** Răspunsul pentru cererile HTTP ușoare, precum verificarea amprentei SHA-256 și interogarea statusului analizei, trebuie să fie returnat în mai puțin de 500 de milisecunde, asigurând flexibilitate interfeței mobile.
- **Timp de execuție asincronă:** Procesul complet de analiză pe server (decompilarea și scanarea fișierelor) ar trebui să se finalizeze, în medie, în 1-3 minute, durata variind strict în funcție de dimensiunea și complexitatea bytecode-ului analizat.
- **Eficiența resurselor pe dispozitivul mobil:** Calculele criptografice și transferul fișierelor trebuie executate exclusiv pe un fir de execuție secundar. Acest lucru garantează că interfața utilizatorului nu se blochează, prevenind erorile de tip ANR (*Application Not Responding*) și menținând consumul de baterie la un nivel minim.

6 Interfața grafică

Interfața grafică a aplicației DroidGuard pune accent pe simplitate, claritate și accesibilitate. Complexitatea tehnică a analizei malware este abstractizată în spatele unui design intuitiv.

6.1 Dashboard-ul principal și selecția aplicațiilor

Ecranul principal oferă utilizatorului o imagine de ansamblu asupra stării de securitate a dispozitivului, urmat de meniul de selecție a aplicațiilor țintă.

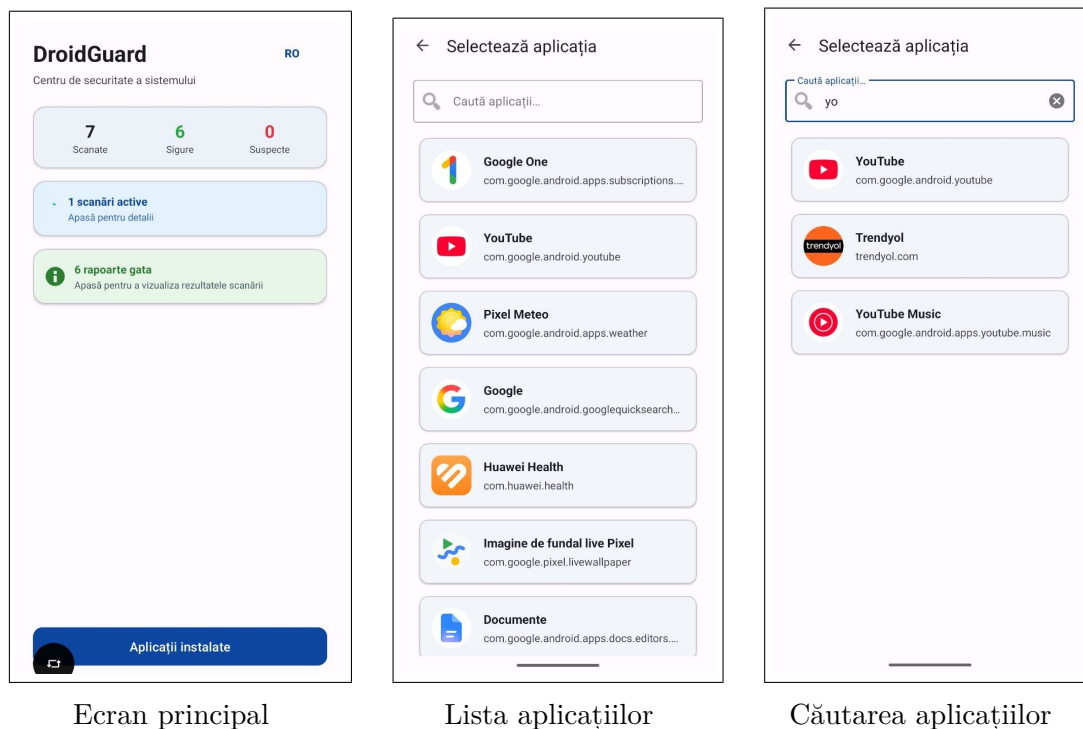


Figura 9: Punctul de intrare în aplicație și selecția aplicației pentru analiză

- **Ecranul principal:** Afășează un panou de statistici globale. Mai jos se află informația despre rapoartele finalizate. Navigarea către începerea unei noi scanări se face prin butonul „Installed apps”, plasat în partea de jos a ecranului.
- **Ecranul de selecție aplicații:** Listează toate aplicațiile instalate pe dispozitiv (cu excepția celor de sistem). Fiecare aplicație este reprezentat printr-un card ce include iconița originală a aplicației, numele public și denumirea tehnică a pachetului. Pentru a identificarea rapidă a aplicației dorite, ecranul de selecție integrează o bară de căutare plasată în partea superioară a listei.

6.2 Fluxul de inițiere, monitorizare și anularea scanării

Pentru a preveni acțiunile accidentale și a oferi control total utilizatorului asupra consumului de date și resurse, sistemul implementează dialoguri explicite de confirmare, și notificări de sistem pentru rularea în fundal.



Figura 10: Mecanismele de control al fluxului de analiză

- **Confirmarea scanării:** La selectarea unei aplicații, un dialog avertizează utilizatorul că datele vor fi trimise către Gateway pentru analiză.
- **Feedback vizual:** Pe durata comunicării cu serverul, interfața afișează un indicator de progres. Această componentă include un buton roșu de anulare.
- **Gestionarea anulării:** La selectarea unei aplicații, un dialog avertizează că scanarea va fi oprită.

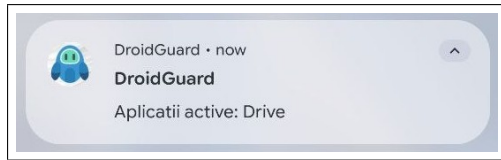
Scanarea și afișarea aplicațiilor se realizează prin următoarele mecanisme:

- **Pornirea cronometrului și schimbarea stării:** La trimiterea inițială a cererii, aplicația se află în starea „În așteptare...” (corespunzătoare stării **PENDING** în coada RabbitMQ). În momentul în care componenta Analysis Worker preia job-ul, serverul transmite prin WebSocket starea **IN_PROGRESS**. Clientul mobil începe un cronometru local, afișând timpul scurs sub forma „Analizare... mm:ss”.
- **Afișarea listei:** Pentru o lizibilitate mai bună, lista din ecranul de monitorizare (Figura 11) folosește un algoritm de sortare pe două niveluri:
 1. **Prioritate după stare:** Aplicația aflată în curs de analiză de Worker este poziționată în capul listei „În curs de scanare”, deasupra aplicațiilor aflate în așteptare.
 2. **Sortare alfabetică:** Aplicațiile aflate în așteptare, precum și lista de „Raporte finalizate”, sunt ordonate alfabetic pentru a facilita localizarea rapidă.

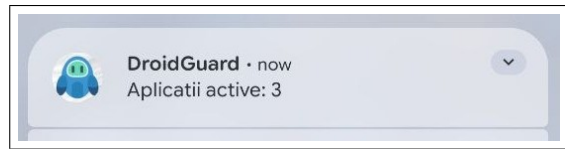


Figura 11: Ecranul principal de monitorizare a scanărilor

- **Execuția în fundal:** Pentru a preveni închiderea conexiunii WebSocket și oprirea cronometrului atunci când utilizatorul părăsește aplicația sau minimizează ecranul, sistemul implementează un serviciu dedicat. Acesta afișează o notificare persistentă în bara de sistem, a cărei informație se adaptează dinamic: dacă se analizează o singură aplicație, se afișează numele acesteia. Dacă sunt scanate mai multe aplicații simultan, se afișează numărul total al acestora. Acest mecanism garantează că procesul de monitorizare continuă fără întreruperi.
- **Notificări la finalizare:** La finalizarea analizei, utilizatorul este anunțat prin sistemul de notificări al telefonului doar dacă nu se află în ecranul de scanare. Sistemul grupează notificările multiple pentru a nu aglomera bara de stare (Figura 12). Un simplu tap pe o notificare extinsă deschide direct raportul de securitate generat pentru aplicația respectivă.



Notificare activă: o singură aplicație



Notificare activă: aplicații multiple



Gruparea notificărilor finalizate



Detalierea notificărilor

Figura 12: Sistemul de notificări pentru monitorizare și finalizare

6.3 Gestionarea erorilor și reîncercarea scanării

Sistemul include un mecanism de gestionare a erorilor ce pot apărea pe parcursul procesului de analiză asincronă (de exemplu, întreruperi ale conexiunii la rețea sau erori de procesare la nivelul componentei Worker).

În cazul în care o scanare eșuează, utilizatorul poate reîncerca trimiterea aplicației, selectând una din cele trei opțiuni:

- **Șterge:** Elimină înregistrarea scanării eșuate din istoricul local și curăță interfața grafică.
- **Anulează:** Închide fereastra de dialog, păstrând starea curentă a aplicației fără a iniția o acțiune suplimentară.
- **Reîncearcă:** Reîncearcă scanarea pentru aplicația respectivă, inițializând o nouă scanare ce va fi trimisă către API Gateway.

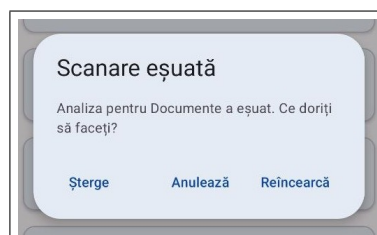


Figura 13: O scanare eșuată

6.4 Gestionarea istoricului și rapoartele de securitate

Rezultatele finale sunt salvate local, eliminând dependența de conexiunea la internet pentru revizualizarea analizelor anterioare.

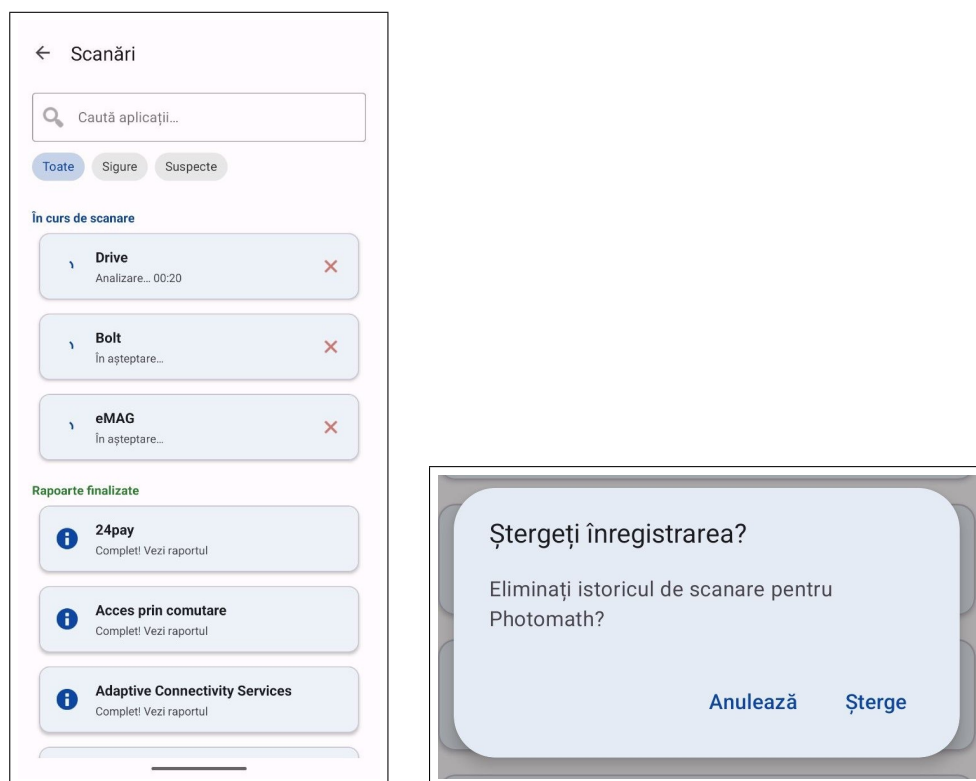
Pentru a facilita navigarea prin istoricul scanărilor și a proceselor active, interfața integrează un antet de control (Figura 14) care conține:

- **Bara de căutare:** Permite utilizatorului să găsească rapid raportul unei anumite aplicații prin introducerea numelui acesteia.
- **Filtre rapide:** Utilizatorul poate filtra dinamic lista pentru a afișa doar aplicațiile „Sigure” sau pe cele „Suspecte”.



Figura 14: Antetul listei de scanări cu opțiuni de căutare și filtrare

Totodată, interfața oferă opțiunea de a elimina istoricul oricărei aplicații analizate anterior prin menținerea apăsată pe cardul aplicației (Figura 15). Ștergerea unei înregistrări din istoricul local va elimina aplicația din listă, însă datele scanate vor rămâne prezente în baza de date globală a sistemului.



Lista rapoartelor salvate

Ștergerea unei înregistrări

Figura 15: Gestionarea istoricului de scanări locale

6.5 Prezentarea rezultatului

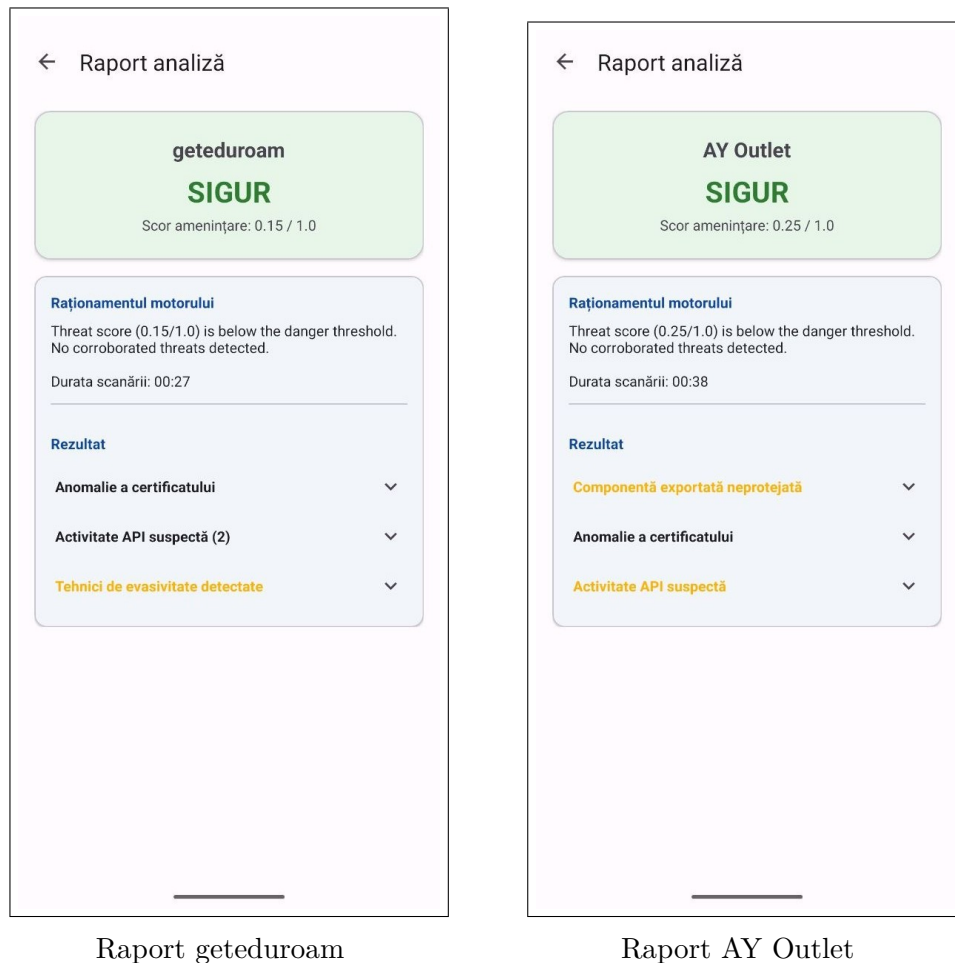


Figura 16: Exemple de rapoarte de analiză detaliate

Fiecare raport este structurat în două secțiuni:

1. **Zona de verdict:** Folosește un cod de culoare pentru fiecare rezultat. Sub verdict este afișat scorul de amenințare normalizat între 0.0 și 1.0.
2. **Raționamentul motorului:** Oferă o explicație textuală a deciziei și un rezumat al indicatorilor descoperiți de backend, cum ar fi amenințările YARA găsite și numărul de euristici și explicația lor.

6.6 Suport multilingv

Aplicația DroidGuard oferă suport bilingv: română și engleză, permițând utilizatorului să schimbe între cele două limbi printr-un buton, accesibil din interfața principală. Schimbarea limbii se aplică asupra tuturor elementelor vizuale ale aplicației.

Figura 17 ilustrează interfața în limba română, iar Figura 18 prezintă aceleași ecrane în limba engleză.

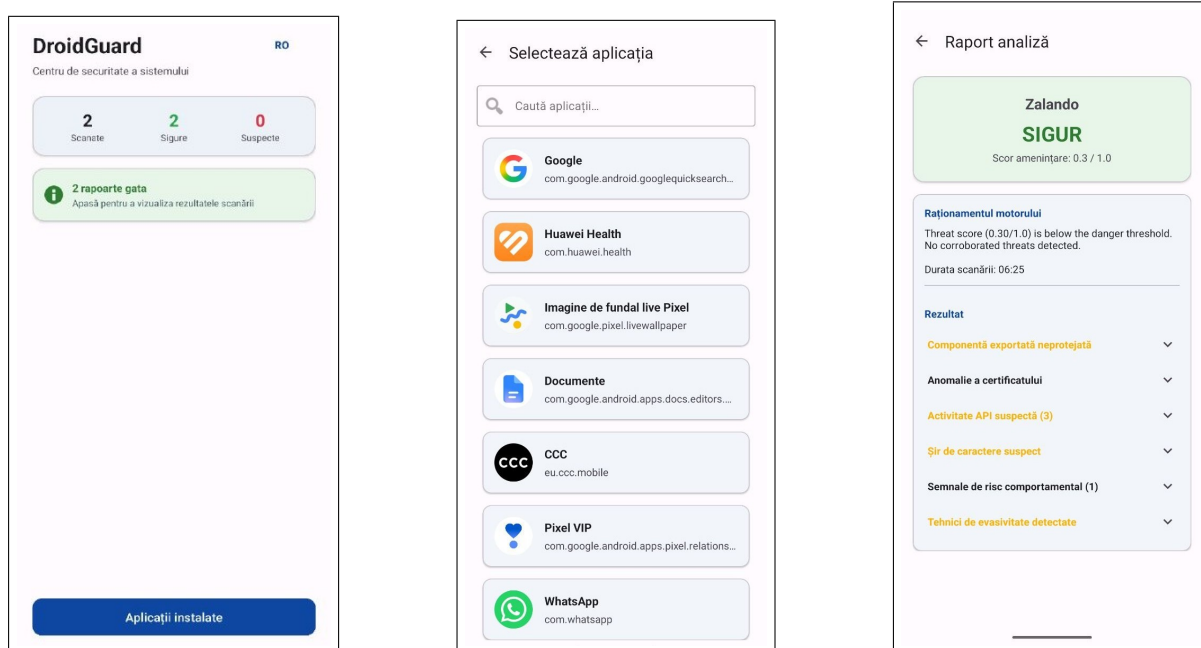


Figura 17: Interfața aplicației în limba română

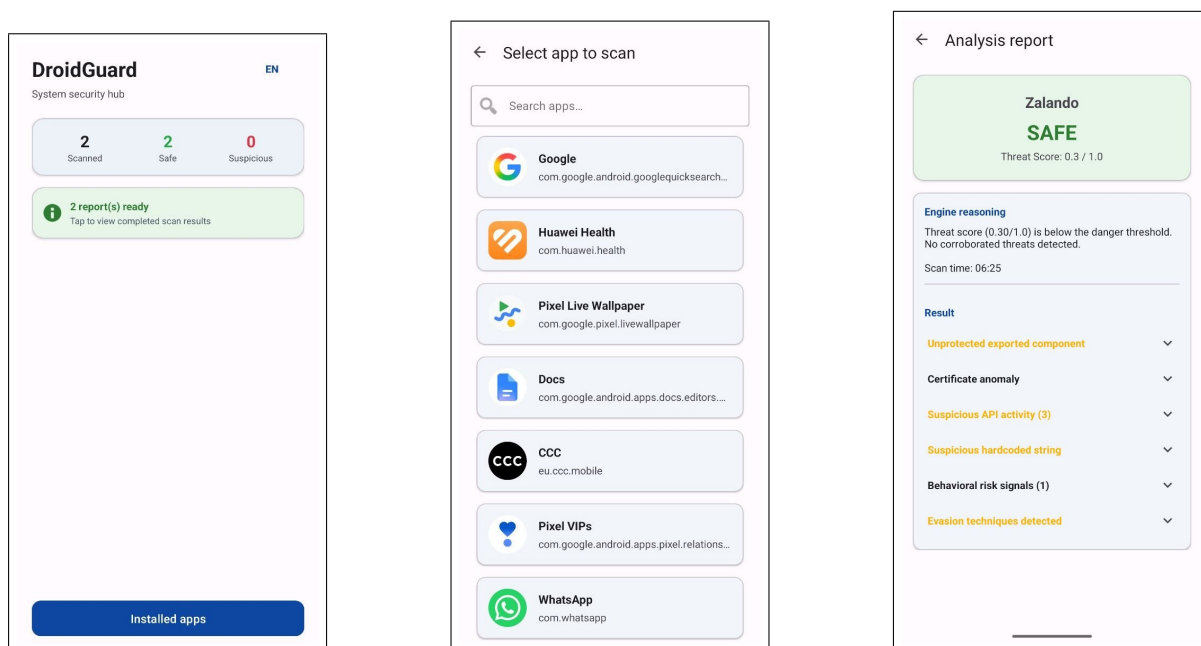


Figura 18: Interfața aplicației în limba engleză

7 Testare

Sistemul a fost testat în două etape distincte: prima pe aplicații comerciale benigne, iar a doua pe aplicații de test construite manual cu grade progresive de comportament malițios. Scopul acestei abordări este de a valida atât capacitatea motorului de a clasifica corect aplicațiile legitime, cât și sensibilitatea la indicatori reali de compromitere.

7.1 Testarea pe aplicații benigne

Sistemul a fost testat pe un set variat de aplicații comerciale standard. Timpul mediu de analiză variază în funcție de aplicație (de obicei, câteva minute). Tabelul 3 sumarizează rezultatele reale obținute în urma testării pe dispozitiv, vizibile și în capturile de ecran.

Aplicație	Verdict	Scorul
geteduroamt	SIGUR	0.15 / 1.0
AY Outlet	SIGUR	0.25 / 1.0

Tabela 3: Rezultate obținute în urma testării a sistemului

7.2 Testarea pe aplicații malițioase construite manual

Pentru a evalua sensibilitatea motorului de decizie, au fost create trei versiuni ale unei aplicații de test HelloWorld, fiecare adăugând progresiv indicatori de compromitere. Toate versiunile partajează aceeași interfață grafică minimală, diferențiindu-se exclusiv prin permisiunile declarate și codul inserat deliberat pentru a simula comportamente malițioase.

Versiunea 1 - Severitate scăzută: Prima versiune declară exclusiv două permisiuni suspecte în `AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.
    RECEIVE_BOOT_COMPLETED"/>
<uses-permission android:name="android.permission.SEND_SMS"/>
```

Aceste permisiuni, deși asociate frecvent cu comportament malițios (pornire automată la boot și trimitere de SMS-uri), nu sunt însoțite de cod care să le exploateze. Motorul a atribuit un scor de **0.15 / 1.0**, confirmând că simpla declarare a permisiunilor, în absența unui cod corespunzător, generează un nivel minim de suspiciune.

Versiunea 2 - Severitate medie: A doua versiune păstrează permisiunile din prima și introduce cod sursă menit să declanșeze euristicile de analiză DEX ale motorului DroidGuard:

```
SmsManager smsManager = SmsManager.getDefault();
smsManager.sendMessage("1234567890", null, "Stealing data!",
    null, null);
```

```
// c3VzcGljaW91cw== reprezentare Base64 al cuvântului suspicious
byte[] decoded = Base64.decode("c3VzcGljaW91cw==", Base64.DEFAULT);
```

Prezența apelului direct către `SmsManager.sendMessage()` cu un mesaj hardcodat, combinată cu decodarea unui șir Base64 (pattern frecvent utilizat pentru ofuscarea payload-urilor), a determinat motorul să crească scorul la **0.30 / 1.0**. Rezultatul reflectă o corelație corectă între permisiunile declarate și utilizarea efectivă a API-urilor sensibile.

Versiunea 3 - Severitate ridicată: Versiunea finală combină toți indicatorii anteriori cu multiple tehnici caracteristice malware-ului real:

```
// Trimitere SMS programatica
SmsManager.getDefault().sendMessage(
    "1234567890", null, "Stealing your money", null, null);

// Tentativa de obtinere acces root
Runtime.getRuntime().exec("su");

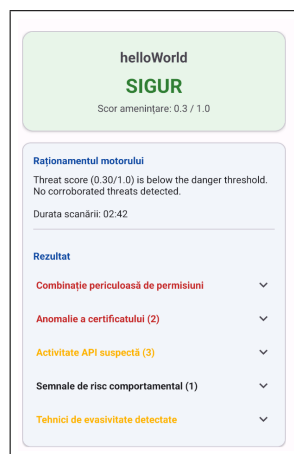
// Incarcare dinamica de cod extern
DexClassLoader("payload.apk", "tmp", null, classLoader);

// URL-uri hardcodate de tip Command and Control
val c2Url = "http://192.168.1.100/command"
val payloadUrl = "http://evil.com/dropper.apk"
```

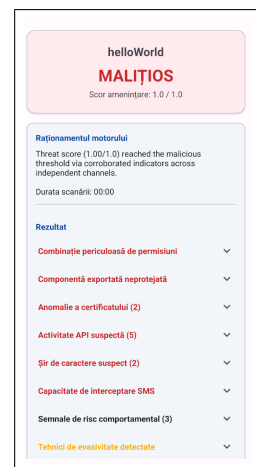
Această versiune activează simultan mai mulți indicatori critici: execuția comenzii `su` (escaladare de privilegii), utilizarea `DexClassLoader` pentru încărcarea dinamică a unui fișier APK extern (tehnica de tip *dropper*), și prezența unor URL-uri hardcodate ce simulează o infrastructură de comandă și control. Motorul a atribuit un scor de **0.91 / 1.0**, clasificând corect aplicația ca fiind periculoasă.



Scor: 0.15 / 1.0



Scor: 0.30 / 1.0



Scor: 1.0 / 1.0

Figura 19: Rapoartele de analiză pentru cele trei versiuni de test cu severitate progresivă

7.3 Sinteza rezultatelor

Tabelul 4 prezintă rezultatele obținute pe cele trei versiuni de test.

Versiune	Verdict	Scorul	Indicatori principali
HelloWorld v1	CLEAN	0.15 / 1.0	Permisii suspecte
HelloWorld v2	CLEAN	0.30 / 1.0	Apeluri SMS + Base64
HelloWorld v3	SUSPICIOUS	1.0 / 1.0	Root, DexClassLoader, C2

Tabela 4: Rezultate obținute pe aplicațiile de test cu severitate progresivă

Progresia scorurilor validează capacitatea motorului de a diferenția între nivelurile de amenințare. În particular, saltul semnificativ la versiunea 3 demonstrează că motorul acordă o pondere majoră tehnicilor avansate de compromitere (escaladare de privilegii, încărcare dinamică de cod, comunicare C2), față de simpla declarare a permisiunilor sau utilizarea izolată a API-urilor sensibile.

7.4 Limitări în analiza aplicațiilor malițioase reale

Deși sistemul a demonstrat rezultate favorabile în mediul de testare, evaluarea unor aplicații malițioase din lumea reală a scos la iveală anumite limitări ale abordării curente. Sistemul analizează comportamentele generice și de suprafață, însă întâmpină dificultăți tehnice majore în momentul în care fișierele APK utilizează tehnici avansate de ofuscare. De exemplu, structurile de cod sunt uneori modificate artificial pentru a forța un nivel de recursivitate extrem de mare în timpul procesului de decompilare, cu scopul de a epuiza resursele alocate.

În plus, procesul de validare și testare practică este mai complicat pe Android. Spre deosebire de alte medii, este complicată găsirea de mostre malware mai vechi, cu arhitecturi simple, care să servească drept punct de plecare pentru evaluare. Majoritatea fișierelor disponibile sunt recente și prezintă un nivel de complexitate ridicat, fiind concepute special pentru a sabota instrumentele de securitate.

Generalizarea unui mecanism de analiză statică pentru a acoperi toate aceste anomalii structurale este un proces extrem de complex, reprezentând o limitare inevitabilă a abordării curente a aplicației.

Pentru a depăși aceste limitări, o soluție optimă pe viitor o reprezintă integrarea tehnicilor de *Machine Learning*. O astfel de abordare ar permite clusterizarea și clasificarea fiecărui tip de aplicație pe baza unor seturi de date reprezentative, permițând sistemului să identifice modele comportamentale complexe în locul aplicării unor reguli rigide. Cu toate acestea, implementarea și antrenarea unui mecanism bazat pe inteligență artificială nu reprezintă scopul principal al acestei practici și depășește aria de acoperire a proiectului curent, motiv pentru care nu a fost integrat în această versiune a sistemului.

8 Concluzii

Proiectul DroidGuard reprezintă o soluție modernă și scalabilă pentru analiza statică de securitate a aplicațiilor Android. Prin adoptarea unei arhitecturi distribuite (Client Mobil, API Gateway și Analysis Worker), sistemul reușește să decupleze interfața utilizatorului de procesarea intensivă necesară decompilării și scanării fișierelor.

Această abordare nu doar că previne suprasolicitarea resurselor hardware ale dispozitivului mobil, dar asigură și un timp de răspuns optim.

În concluzie, DroidGuard oferă utilizatorilor un instrument accesibil pentru identificarea potențialelor amenințări, ascunzând complexitatea tehnică în spatele unei interfețe grafice intuitive și pune o bază solidă pentru viitoarele dezvoltări în domeniul securității cibernetice mobile.

Bibliografie

- [1] Editare documentație:
<https://www.overleaf.com>
<https://mermaid.live>
<https://www.drawio.com>

- [2] Logo:
https://www.flaticon.com/free-icon/robot_3558977?term=android&page=1&position=23&origin=tag&related_id=3558977

- [3] Documentație proiect:
<https://supabase.com>
<https://www.rabbitmq.com/client-libraries/java-api-guide#jdk-versions>
<https://www.youtube.com/watch?v=4-xApAPhxJQ>
<https://spring.io>
<https://developer.android.com/develop>
<https://docs.docker.com>
<https://www.postgresql.org/docs/>

- [4] Detectie anomalii în dispozitive mobile:
<https://attack.mitre.org/matrices/mobile/>
<https://attack.mitre.org/techniques/T1518/001/>
[https://developer.android.com/reference/android/telephony/TelephonyManager#getLine1Number\(\)](https://developer.android.com/reference/android/telephony/TelephonyManager#getLine1Number())
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7958624>
https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Enck.pdf
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10128288>
https://media.telefonicatech.com/telefonicatech/uploads/2021/1/4915_2014-ndss.pdf
<https://www.abartel.net/static/p/tr-2013-taintandroid.pdf>
<https://www.bodden.de/pubs/far+14flowdroid.pdf>
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8886364>
<https://malpedia.caad.fkie.fraunhofer.de>
<https://github.com/Pithus>
<https://github.com/mobsf/mobile-security-framework-mobsf>

- [5] Documentație Android:
<https://pysmali.readthedocs.io/en/latest/api/smali/index.html>
<https://source.android.com/docs/core/runtime/dex-format>
<https://developer.android.com/reference/kotlin/classes>

- [6] Informații Yara:
<https://yara.readthedocs.io/en/latest/writingrules.html#conditions>
<https://github.com/yara-rules/rules>
<https://github.com/Koodous/rules>
<https://github.com/pedramamini/awesome-yara#rules>
<https://github.com/abusech/YARAify>
<https://github.com/anyrun/YARA>
<https://github.com/mikesxrs/Open-Source-YARA-rules>

- [7] Androguard:
<https://androguard.readthedocs.io/en/latest/>
<https://github.com/androguard/androguard>